



Inteligencia en redes de datos

BIT-31

Proyecto Final

“Plataforma inteligente para clasificación y atención de incidentes de red, integrando un modelo local con Ollama, RAG sobre procedimientos técnicos, un servidor MCP para consultar métricas, inventario y tiquetes, y mecanismos de evaluación para medir precisión, latencia, costo y alucinaciones”

Autor:

Alejandro Moran Reyes (202401114359)

Kendall Valerin Solano (202401100994)

Profesor:

Ing. Carlos Andrés Méndez Rodríguez

San José, 19 de agosto 2026

Introducción

La creciente complejidad de las redes de datos modernas ha incrementado significativamente la cantidad de incidentes que deben ser atendidos diariamente por los equipos de soporte técnico. Las organizaciones dependen de infraestructuras compuestas por dispositivos físicos, máquinas virtuales, servicios en la nube y aplicaciones distribuidas, donde cualquier falla puede afectar la disponibilidad de los servicios y provocar pérdidas operativas. En este contexto, la clasificación rápida y correcta de los incidentes constituye un elemento fundamental para reducir los tiempos de respuesta y garantizar la continuidad de las operaciones.

Tradicionalmente, la gestión de incidentes depende del conocimiento y experiencia del personal técnico, quien debe interpretar la descripción del problema, identificar los equipos involucrados, consultar procedimientos técnicos, revisar métricas de funcionamiento y determinar las acciones más apropiadas para resolver la situación. Este proceso suele ser repetitivo, consume una cantidad considerable de tiempo y puede generar errores cuando existen altos volúmenes de solicitudes o cuando la información disponible es insuficiente.

Los avances recientes en inteligencia artificial, particularmente en los Modelos de Lenguaje Grandes (LLM), han abierto nuevas posibilidades para automatizar tareas relacionadas con la interpretación del lenguaje natural. Sin embargo, un modelo de lenguaje utilizado de forma aislada presenta limitaciones importantes, ya que únicamente responde con base en el conocimiento adquirido durante su entrenamiento y no tiene acceso directo a la información actualizada de la infraestructura tecnológica de una organización. Como consecuencia, puede producir respuestas imprecisas o generar información inexistente, fenómeno conocido como alucinación.

Para reducir estas limitaciones, durante los últimos años han surgido arquitecturas que complementan los modelos de lenguaje mediante mecanismos externos de consulta. Entre ellas destaca **Retrieval-Augmented Generation (RAG)**, una técnica que permite recuperar documentos relevantes antes de generar la respuesta, utilizando información contenida en manuales técnicos, procedimientos institucionales o bases documentales. De esta manera,

las respuestas producidas por el modelo se fundamentan en información verificable y actualizada.

De forma complementaria, el **Model Context Protocol (MCP)** permite conectar el modelo de lenguaje con herramientas externas capaces de consultar bases de datos, inventarios de dispositivos, sistemas de monitoreo y plataformas de gestión de incidentes. Gracias a este protocolo, el modelo puede acceder dinámicamente a información reciente sin necesidad de volver a entrenarse, facilitando diagnósticos más precisos y mejor fundamentados.

En este proyecto se propone el diseño de una plataforma inteligente para la clasificación y atención de incidentes de red que integre un modelo local ejecutado mediante Ollama, una arquitectura RAG para consultar procedimientos técnicos y un servidor MCP encargado de proporcionar acceso a métricas, inventario y tickets de soporte. Asimismo, se plantea evaluar el desempeño de la solución mediante indicadores como precisión de clasificación, latencia, costo computacional y nivel de alucinaciones, con el propósito de determinar la viabilidad del uso de modelos locales en entornos de administración de redes.

Definición del tema

El presente proyecto propone el diseño e implementación de una plataforma inteligente orientada a la clasificación y atención de incidentes de red mediante la integración de tecnologías modernas de inteligencia artificial. La solución utilizará un modelo de lenguaje ejecutado localmente mediante Ollama, una arquitectura Retrieval-Augmented Generation (RAG) para consultar procedimientos técnicos institucionales y un servidor basado en Model Context Protocol (MCP) para acceder a información actualizada proveniente de métricas de red, inventarios de dispositivos y sistemas de gestión de tiquetes.

A diferencia de las soluciones tradicionales, que únicamente clasifican incidentes utilizando palabras clave o reglas predefinidas, la plataforma propuesta busca comprender la descripción del problema escrita en lenguaje natural, recuperar información técnica relevante y consultar herramientas externas para generar una recomendación fundamentada sobre la atención del incidente.

El sistema permitirá evaluar el comportamiento del modelo considerando aspectos como precisión de clasificación, tiempo de respuesta, consumo de recursos computacionales, costo operativo y frecuencia de alucinaciones, con el objetivo de analizar la viabilidad de utilizar modelos locales de inteligencia artificial dentro de procesos de administración de redes.

Planteamiento del problema

Los departamentos encargados de administrar redes de datos reciben diariamente incidentes relacionados con fallos de conectividad, degradación del servicio, problemas de autenticación, errores de configuración, interrupciones de aplicaciones y múltiples eventos asociados a la infraestructura tecnológica. La atención de estos incidentes suele depender del conocimiento del personal especializado, quien debe interpretar la descripción del problema, consultar documentación técnica, verificar el estado de los dispositivos involucrados y decidir las acciones necesarias para resolver la situación.

Este procedimiento presenta varias limitaciones. En primer lugar, la clasificación manual consume tiempo y dificulta la atención rápida cuando existe un elevado volumen de solicitudes. Además, la información requerida para diagnosticar un incidente normalmente se encuentra distribuida en diferentes plataformas, como inventarios de equipos, sistemas de monitoreo, bases de conocimiento y gestores de tiquetes, lo que obliga al personal técnico a consultar múltiples fuentes antes de emitir una recomendación.

Aunque los modelos de lenguaje representan una alternativa prometedora para automatizar este proceso, su utilización de forma independiente presenta inconvenientes importantes. Estos modelos pueden generar respuestas incorrectas cuando no poseen información actualizada o cuando inventan datos inexistentes, fenómeno conocido como alucinación. Asimismo, las soluciones basadas exclusivamente en servicios en la nube pueden representar riesgos relacionados con la privacidad, el costo y la dependencia de proveedores externos.

Por estas razones surge la necesidad de investigar una arquitectura que combine un modelo local ejecutado mediante Ollama con mecanismos de recuperación de información y herramientas externas que permitan acceder a datos actualizados de la infraestructura de red, incrementando la confiabilidad de las respuestas y reduciendo las limitaciones propias de los modelos de lenguaje utilizados de manera aislada.

Justificación

La aplicación de inteligencia artificial en la administración de redes constituye una de las principales tendencias dentro del área de telecomunicaciones y computación. La posibilidad de automatizar procesos de clasificación y atención de incidentes permite reducir tiempos de respuesta, mejorar la calidad del soporte técnico y optimizar el uso de los recursos humanos disponibles.

El uso de modelos locales ejecutados mediante Ollama ofrece ventajas importantes relacionadas con la privacidad de la información, ya que evita enviar datos sensibles hacia servicios externos. Esta característica resulta especialmente relevante para instituciones que administran infraestructura crítica o información confidencial.

La incorporación de una arquitectura RAG permitirá que las respuestas generadas por el modelo se fundamenten en procedimientos técnicos previamente documentados, disminuyendo la probabilidad de producir recomendaciones incorrectas o inventadas. Por otra parte, el uso de un servidor MCP facilitará la consulta de métricas, inventario y sistemas de tickets, proporcionando al modelo acceso a información actualizada durante el proceso de inferencia.

Finalmente, la evaluación del sistema mediante métricas objetivas como precisión, latencia, costo computacional y frecuencia de alucinaciones permitirá determinar si este tipo de arquitectura representa una alternativa viable para apoyar las operaciones de gestión de redes dentro de organizaciones modernas.

Objetivo general

Diseñar e implementar una plataforma inteligente para la clasificación y atención de incidentes de red mediante la integración de un modelo local ejecutado con Ollama, una arquitectura Retrieval-Augmented Generation (RAG) para la consulta de procedimientos técnicos y un servidor Model Context Protocol (MCP) para acceder a métricas, inventario y sistemas de tiquetes, evaluando su desempeño mediante indicadores de precisión, latencia, costo computacional y alucinaciones.

Objetivos específicos

- Diseñar la arquitectura general de una plataforma inteligente para la clasificación y atención de incidentes de red.
- Implementar un modelo de lenguaje local utilizando Ollama como motor de inferencia.
- Desarrollar una base documental para RAG que permita recuperar procedimientos técnicos durante la generación de respuestas.
- Implementar un servidor MCP capaz de consultar métricas de red, inventario de dispositivos y sistemas de gestión de tiquetes.
- Integrar los componentes de la plataforma para apoyar el diagnóstico y la atención de incidentes de red.
- Evaluar el desempeño del sistema mediante indicadores de precisión, latencia, costo computacional, consumo de recursos y frecuencia de alucinaciones.

Planificación del proyecto

El desarrollo de la plataforma inteligente propuesta se realizará mediante una metodología incremental, permitiendo construir y validar cada componente de manera independiente antes de integrarlos en una solución completa. Esta estrategia facilita la identificación temprana de errores, reduce la complejidad del proceso de implementación y permite realizar ajustes conforme avanza la investigación.

La planificación contempla inicialmente la preparación del entorno de desarrollo, seguido de la instalación de las herramientas necesarias para ejecutar modelos de lenguaje de forma local. Posteriormente se desarrollará la arquitectura RAG encargada de recuperar procedimientos técnicos, así como el servidor MCP que permitirá acceder a información dinámica de la infraestructura de red. Finalmente, todos los componentes serán integrados para realizar experimentos orientados a medir el rendimiento y la calidad de las respuestas generadas.

El proyecto se ejecutará utilizando una máquina virtual con sistema operativo Ubuntu sobre WSL2, empleando Python como lenguaje principal de programación debido a su amplia disponibilidad de bibliotecas relacionadas con inteligencia artificial, procesamiento de datos y automatización.

Metodología de investigación

La presente investigación posee un enfoque **aplicado**, ya que busca resolver un problema real relacionado con la gestión de incidentes de red mediante la implementación de una solución tecnológica basada en inteligencia artificial.

Asimismo, corresponde a una investigación de tipo **experimental**, debido a que se desarrollará un prototipo funcional cuya finalidad será evaluar el comportamiento de diferentes componentes de inteligencia artificial bajo condiciones controladas. Durante las pruebas se analizará el desempeño del sistema utilizando un conjunto de incidentes de red previamente definidos, permitiendo medir objetivamente indicadores de calidad y rendimiento.

La metodología se divide en cinco etapas principales:

Etapla 1. Diseño de la arquitectura

En esta etapa se definirá la arquitectura general de la plataforma, estableciendo la comunicación entre el modelo local ejecutado mediante Ollama, el sistema RAG encargado de recuperar procedimientos técnicos y el servidor MCP responsable de consultar métricas, inventario y tiquetes.

Además, se determinarán las interfaces necesarias para que todos los componentes puedan intercambiar información de forma segura y eficiente.

Etapla 2. Implementación del entorno

Posteriormente se configurará el entorno de desarrollo sobre Ubuntu utilizando WSL2, instalando las herramientas necesarias para la ejecución del proyecto.

Entre las actividades principales se encuentran:

- Instalación de Python.
- Instalación de Ollama.

- Descarga del modelo de lenguaje seleccionado.
- Configuración de PostgreSQL.
- Instalación de FastMCP.
- Configuración del repositorio documental para RAG.
- Instalación de bibliotecas de procesamiento de lenguaje natural.

El objetivo de esta etapa consiste en disponer de un ambiente completamente funcional que permita desarrollar y ejecutar todos los componentes de la plataforma.

Etapa 3. Integración de los componentes

Una vez preparado el entorno, se procederá a integrar cada uno de los módulos desarrollados.

El flujo general del sistema será el siguiente:

Incidente recibido

↓

Aplicación en Python

↓

Clasificación mediante Ollama



Consulta de documentos utilizando RAG



Consulta del servidor MCP



Obtención de:

- Inventario
- Métricas
- Historial de tiquetes



Generación de respuesta



Evaluación de resultados

Durante esta etapa se verificará que el modelo pueda utilizar simultáneamente la información obtenida desde la base documental y desde las herramientas expuestas mediante MCP.

Etapa 4. Experimentación

Finalizada la integración se realizarán diferentes pruebas utilizando incidentes de red previamente seleccionados.

Cada incidente será procesado por la plataforma con el propósito de determinar:

- Categoría asignada.
- Procedimiento técnico recuperado.
- Información obtenida mediante MCP.
- Respuesta generada por el modelo.
- Tiempo total de procesamiento.
- Consumo de recursos computacionales.

Los resultados obtenidos serán almacenados para su posterior análisis estadístico.

Etapas 5. Evaluación

La última etapa consistirá en analizar el comportamiento general del sistema mediante diferentes indicadores cuantitativos.

Los resultados permitirán comparar la calidad de las respuestas generadas y determinar la viabilidad del uso de modelos locales dentro de procesos de administración de redes.

Herramientas y recursos

Para el desarrollo del proyecto se utilizarán herramientas de software libre y plataformas ampliamente utilizadas en el ámbito de la inteligencia artificial y la administración de redes.

Herramienta	Función dentro del proyecto
Ubuntu (WSL2)	Sistema operativo utilizado para el desarrollo del proyecto.
Python	Lenguaje principal para implementar la plataforma.
Visual Studio Code	Entorno de desarrollo integrado (IDE).
Ollama	Ejecución local del modelo de lenguaje.
FastMCP	Implementación del servidor MCP.
PostgreSQL	Almacenamiento de inventario, métricas y tickets.
ChromaDB o FAISS	Base de datos vectorial utilizada por RAG.
Sentence Transformers	Generación de embeddings para la búsqueda semántica.
Pandas	Manipulación de conjuntos de datos.
Psutil	Monitoreo de CPU y memoria durante las pruebas.
Matplotlib	Generación de gráficos para el análisis de resultados.
GitHub	Control de versiones y almacenamiento del proyecto.

Recursos de información

La plataforma utilizará diferentes fuentes de información para generar respuestas fundamentadas.

Entre ellas se encuentran:

- Procedimientos técnicos institucionales.
- Manuales de configuración de dispositivos de red.
- Inventario de equipos.
- Base de datos de incidentes históricos.
- Métricas obtenidas desde herramientas de monitoreo.
- Catálogo de servicios de tecnología.
- Documentación oficial de fabricantes.

Toda esta información será utilizada mediante mecanismos de recuperación controlada, evitando que el modelo genere respuestas basadas únicamente en el conocimiento adquirido durante su entrenamiento.

Variables de evaluación

Con el propósito de determinar el desempeño del sistema se utilizarán los siguientes indicadores:

Variable	Descripción
Precisión	Porcentaje de incidentes correctamente clasificados.
Latencia	Tiempo requerido para generar una respuesta.
Throughput	Cantidad de incidentes procesados por unidad de tiempo.
Uso de CPU	Consumo promedio del procesador durante la inferencia.
Uso de memoria	Memoria RAM utilizada por el modelo durante la ejecución.
Costo computacional	Recursos requeridos para ejecutar el sistema localmente.
Alucinaciones	Cantidad de respuestas que contienen información incorrecta o inventada.

Resultados esperados

Se espera desarrollar un prototipo funcional capaz de clasificar incidentes de red utilizando un modelo local ejecutado mediante Ollama, consultar procedimientos técnicos mediante una arquitectura RAG y acceder a información dinámica utilizando un servidor MCP.

Además, se espera que la integración de estas tecnologías permita reducir la frecuencia de respuestas incorrectas, mejorar la precisión de clasificación y disminuir los tiempos requeridos para apoyar el proceso de atención de incidentes.

Finalmente, la investigación pretende demostrar que el uso de modelos locales constituye una alternativa viable para organizaciones que requieren preservar la privacidad de la información y reducir la dependencia de servicios externos de inteligencia artificial, manteniendo niveles adecuados de rendimiento y confiabilidad.

Marco de referencia

La revisión bibliográfica del presente proyecto se organiza en cinco ejes temáticos que sustentan el diseño de una plataforma inteligente para la clasificación y atención de incidentes de red mediante la integración de modelos de lenguaje locales, recuperación aumentada por generación (RAG) y el protocolo Model Context Protocol (MCP). Estos ejes permiten comprender las tecnologías involucradas, sus ventajas, limitaciones y los criterios que serán utilizados para evaluar el desempeño del sistema propuesto.

Eje 1: Inteligencia Artificial aplicada a la gestión de incidentes de red

La inteligencia artificial ha experimentado un crecimiento significativo en los últimos años gracias al desarrollo de modelos capaces de comprender lenguaje natural y asistir en tareas complejas de análisis. Dentro del ámbito de la administración de redes, estas tecnologías permiten automatizar procesos relacionados con la clasificación de incidentes, el diagnóstico preliminar y la generación de recomendaciones para el personal técnico.

Los centros de soporte reciben diariamente una gran cantidad de solicitudes relacionadas con problemas de conectividad, fallos de autenticación, errores de configuración, degradación del rendimiento y pérdida de disponibilidad de servicios. Tradicionalmente estas solicitudes son clasificadas manualmente por operadores especializados, quienes deben interpretar la descripción del incidente y determinar la categoría correspondiente.

Diversas investigaciones recientes demuestran que los modelos de lenguaje pueden comprender este tipo de descripciones escritas en lenguaje natural y asignarlas automáticamente a categorías predefinidas con niveles elevados de precisión, reduciendo considerablemente los tiempos de respuesta y mejorando la eficiencia operativa.

Sin embargo, estos modelos también presentan limitaciones importantes cuando no cuentan con acceso a información actualizada sobre la infraestructura tecnológica, situación que motiva la integración con arquitecturas RAG y herramientas MCP.

Eje 2: Modelos de lenguaje locales mediante Ollama

Los modelos de lenguaje grandes (LLM) representan una de las tecnologías más importantes dentro del desarrollo reciente de la inteligencia artificial. Estos modelos son entrenados utilizando grandes cantidades de información y posteriormente pueden generar respuestas en lenguaje natural para múltiples tareas.

Aunque existen numerosos proveedores comerciales que ofrecen acceso mediante servicios en la nube, en muchas organizaciones resulta necesario ejecutar estos modelos localmente debido a políticas de privacidad, confidencialidad y soberanía de datos.

Ollama constituye una plataforma que facilita la descarga, administración y ejecución de modelos de lenguaje directamente sobre equipos locales. Esta herramienta permite utilizar diferentes modelos de código abierto sin depender de servicios externos, ofreciendo una interfaz sencilla para integrarlos con aplicaciones desarrolladas en Python.

La ejecución local también reduce costos asociados al consumo de servicios comerciales y proporciona un mayor control sobre la infraestructura utilizada para realizar las inferencias.

No obstante, el desempeño del sistema dependerá de los recursos disponibles, especialmente memoria RAM, capacidad del procesador y tamaño del modelo seleccionado.

Eje 3: Retrieval-Augmented Generation (RAG)

Retrieval-Augmented Generation (RAG) es una arquitectura diseñada para mejorar las respuestas generadas por los modelos de lenguaje mediante la incorporación de información recuperada desde documentos externos.

En lugar de responder únicamente utilizando el conocimiento adquirido durante el entrenamiento, el sistema realiza primero una búsqueda semántica dentro de una base documental para localizar la información más relevante relacionada con la consulta realizada por el usuario.

En el contexto de este proyecto, la base documental estará conformada por procedimientos técnicos, manuales de configuración, guías de solución de problemas, documentación institucional y otros recursos utilizados habitualmente por el personal de soporte.

Posteriormente, los documentos recuperados serán enviados junto con la consulta al modelo de lenguaje, permitiendo generar respuestas fundamentadas y reduciendo significativamente la probabilidad de producir información incorrecta o inventada.

Esta arquitectura resulta especialmente útil en entornos donde la información cambia constantemente y donde es necesario consultar documentación actualizada sin volver a entrenar el modelo.

Eje 4: Model Context Protocol (MCP)

El Model Context Protocol (MCP) constituye un protocolo abierto diseñado para permitir que los modelos de lenguaje interactúen con herramientas externas de forma estandarizada.

Gracias a MCP, un modelo puede consultar diferentes fuentes de información durante la generación de respuestas, incluyendo bases de datos, servicios web, inventarios, sistemas de monitoreo, gestores de incidentes y múltiples aplicaciones empresariales.

En este proyecto el servidor MCP proporcionará acceso a:

- Inventario de dispositivos de red.
- Métricas de rendimiento.
- Estado de interfaces.
- Historial de incidentes.
- Catálogo de servicios.
- Información de tiquetes.

Esta capacidad permitirá que el modelo utilice información actualizada en tiempo real para apoyar el proceso de clasificación y atención de incidentes. Asimismo, el uso de MCP facilita la separación entre el modelo de lenguaje y las fuentes de información, mejorando la seguridad y simplificando el mantenimiento de la solución.

Eje 5: Evaluación del desempeño del sistema

La evaluación constituye una parte fundamental del proyecto, ya que permitirá determinar objetivamente la calidad de la plataforma desarrollada.

Los principales indicadores considerados serán:

Precisión

Mide el porcentaje de incidentes correctamente clasificados respecto a una muestra previamente etiquetada.

Latencia

Corresponde al tiempo requerido desde que el usuario envía un incidente hasta que el sistema genera una respuesta.

Throughput

Representa la cantidad de incidentes procesados por unidad de tiempo durante las pruebas experimentales.

Costo computacional

Considera el consumo de recursos necesarios para ejecutar el modelo localmente, incluyendo memoria RAM, utilización del procesador y tiempo de ejecución.

Uso de recursos

Evalúa el comportamiento del sistema respecto al consumo de CPU y memoria durante la inferencia.

Alucinaciones

Hace referencia a la generación de respuestas incorrectas o inventadas por parte del modelo. En este proyecto se verificará si las recomendaciones emitidas corresponden

efectivamente a la información recuperada mediante RAG y a los datos proporcionados por MCP.

Síntesis del marco de referencia

La literatura analizada demuestra que la combinación de modelos de lenguaje locales, recuperación aumentada por generación y herramientas externas constituye una estrategia prometedora para mejorar la administración de incidentes de red.

Mientras Ollama proporciona una plataforma eficiente para ejecutar modelos localmente, RAG permite fundamentar las respuestas utilizando documentación técnica actualizada y MCP facilita el acceso dinámico a información proveniente de la infraestructura tecnológica.

La integración de estas tecnologías busca reducir el tiempo necesario para clasificar incidentes, mejorar la precisión de las recomendaciones y disminuir la probabilidad de producir respuestas incorrectas, manteniendo además un adecuado nivel de privacidad y control sobre la información institucional.

Referencias bibliográficas

- Ekinci, E., & Öztürk, Ş. (2024). *IT service desk ticket classification via large language models*. En *2024 32nd Signal Processing and Communications Applications Conference (UBMK)*. IEEE. <https://doi.org/10.1109/UBMK63289.2024.10773473>
- Husom, E. J., Goknil, A., Astekin, M., Shar, L. K., Kåsen, A., Sen, S., Mithassel, B. A., & Soyly, A. (2025). *Sustainable LLM inference for edge AI: Evaluating quantized LLMs for energy efficiency, output accuracy, and inference latency*. arXiv. <https://arxiv.org/abs/2504.03360>
- Liu, F., Chen, J., Zhang, H., He, X., Li, Y., Wu, G., Zhang, T., Yi, L., & Shi, R. (2025). *TickIt: Leveraging large language models for automated ticket escalation*. arXiv. <https://arxiv.org/abs/2504.08475>
- CEUR Workshop Proceedings. (2023). *LLM-based approaches for automatic ticket assignment*. <https://ceur-ws.org/Vol-3551/paper4.pdf>
- Benchmarking large language model inference on limited-resource edge systems*. (2026). *Electronics*, 15(11), 2451. <https://doi.org/10.3390/electronics15112451>
- Anthropic. (2024). *Model Context Protocol (MCP) Specification*. <https://modelcontextprotocol.io/>
- Ollama. (2025). *Ollama Documentation*. <https://ollama.com/>
- Lewis, P., Perez, E., Piktus, A., et al. (2020). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. <https://arxiv.org/abs/2005.11401>
- PostgreSQL Global Development Group. (2025). *PostgreSQL Documentation*. <https://www.postgresql.org/docs/>
- Python Software Foundation. (2025). *Python Documentation*. <https://docs.python.org/3/>